TN-IMPACT 2026

IDENTIFYING INDUSTRIAL CHALLENGES

PROBLEM STATEMENT NUMBER

TNI26154

PROBLEM STATEMENT TITLE:

AI for Safety Gear Detection: Build a computer vision model that uses site CCTV feeds to detect workers not wearing helmets or harnesses in real-time and automatically logs the safety violation.

Submitted by

NAME	REGISTRATION NUMBER

Certificate

This report is submitted by Team **…**, consisting of students pursuing …, under the guidance of …, as part of the TN-IMPACT 2026 Hackathon.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our mentor …., for providing consistent technical guidance throughout the development of this project. We thank the TN-IMPACT 2026 organizing committee for presenting this opportunity to address real-world industrial safety challenges. We also acknowledge the open-source communities behind Ultralytics YOLOv8, FastAPI, and OpenCV, whose tools formed the backbone of our implementation.

TABLE OF CONTENTS

S.NO	TITLE	PAGE NUMBER
1	ABSTRACT	6
2	INTRODUCTION	6
3	LITERATURE REVIEW	7
4	PROBLEM STATEMENT-Design methodology	8
5	Development of SYSTEM / prototype	9
6	Design validation/ Prototype testing	12
7	RESULTS AND DISCUSSIONS	15
8	FUTURE SCOPE	16
9	CONCLUSION	17
10	PROJECT ARTIFACTS & ACCESS LINKS	17

LIST OF TABLES

S.NO	DESCRIPTION	PAGE NUMBER
1	Design Decision Rationale Table	8
2	Test Case Validation Matrix	12
3	System Performance Metrics	14

LIST OF FIGURES

S.NO	DESCRIPTION	PAGE NUMBER
1	System Architecture — PPE Detection Pipeline	11
2	Sample Detection Frame — Helmet and Vest Detected	12
3	Sample Detection Frame — Helmet Violation Flagged	12
4	Sample Detection Frame — Vest Violation Flagged	13
5	Two persons, one compliant, one not compliant	13
6	Empty frame (no persons)	14

1. ABSTRACT

Industrial worksites — particularly **construction zones** — record alarmingly high rates of preventable injuries caused by non-compliance with Personal Protective Equipment (PPE) mandates. Manual supervision is inconsistent, expensive, and unable to scale across large sites with multiple workers. This project presents an AI-powered, real-time PPE compliance detection system built on the YOLOv8 object detection framework, exposed as a FastAPI-based REST API and integrated with an interactive frontend monitoring interface. The system processes individual video frames submitted from site cameras through the frontend, detects persons and their associated PPE items (helmets and safety vests), identifies violations per individual, and returns structured violation reports per frame. A RiskEngine module records time-stamped violations per worker to enable future risk-score computation. The utility layer provides geometric functions for accurate object-to-person association. The combined frontend–backend architecture is designed as a lightweight, deployable solution that enables real-time monitoring, alert visualization, and compliance logging in industrial environments.

2. INTRODUCTION

Workplace safety in industrial and construction environments is governed by strict PPE compliance regulations. Despite mandates, enforcement remains largely dependent on human supervisors, which introduces inconsistency, human fatigue, and coverage gaps — especially across large multi-worker sites operating in shifts.

Computer vision has emerged as a transformative technology for automating safety surveillance. Modern object detection architectures, particularly the YOLO (You Only Look Once) family, enable real-time, high-accuracy detection of objects within image frames at speeds suitable for live video analysis.

This project builds an end-to-end PPE detection pipeline comprising five modules:

1. **PPEDetector:** A YOLOv8-based detection engine that identifies persons, helmets, vests, and explicit no-helmet events in each submitted frame.
2. **Frontend Interface:** A real-time monitoring dashboard that captures video frames from camera feeds, displays live detection results, and visualizes violations for operators.
3. **FastAPI Backend:** Exposes a /analyse endpoint that accepts image frames from the frontend and returns structured per-frame violation reports.
4. **RiskEngine:** Tracks per-person violation history with timestamps for temporal risk analysis and future risk-score computation.

5. **Utility Module:** Provides geometric functions for bounding-box overlap and PPE-to-person association logic to accurately assign detected equipment to the correct individual.

The system targets construction sites, manufacturing floors, and other industrial environments where helmet and vest compliance is mandatory, offering a scalable and automated alternative to manual safety monitoring.

3. LITERATURE REVIEW

3.1 Object Detection in Safety Applications

YOLO-based architectures (Redmon et al., 2016 onward) have been widely adopted for real-time PPE detection due to their single-pass inference design, which achieves near-video-frame-rate detection on standard GPUs. YOLOv8 (Ultralytics, 2023) further improves anchor-free detection and transfer learning support, making it ideal for domain-specific fine-tuning on PPE datasets.

3.2 PPE Detection Datasets

Publicly available datasets containing annotated classes such as helmets, vests, gloves, and persons have enabled supervised training of PPE detectors(DATASET LINK) The model (best.pt) used in this project is trained on a dataset with at minimum the following class indices: 0 (helmet), 2 (vest), 6 (person), and 7 (no_helmet).

3.3 Person-Object Association

Associating detected PPE items with specific individuals checks whether a helmet's centroid lies within the upper half of a person's bounding box - provide a lightweight, inference free association method suitable for real time systems.

3.4 Risk Scoring in Safety Systems

Responsible for maintaining a per-person violation history. Each instance is assigned to one detected worker and records every PPE violation that worker commits along with the exact Unix timestamp of the event.

4. PROBLEM STATEMENT – DESIGN METHODOLOGY

4.1 Problem Definition

Manual PPE compliance monitoring on construction and industrial sites is reactive, inconsistent, and cannot scale. The challenge is to build an automated system that can:
- Detect individual workers in a scene
- Determine, per worker, whether mandatory PPE (helmet, safety vest) is present
- Log violations with temporal context
- Operate in near real-time from camera feed frames

4.2 Design Approach

The system follows a modular pipeline architecture:

[Camera Frame Input]
↓
[PPEDetector — YOLOv8 Inference]
↓
[Object Classification: person / helmet / vest / no_helmet]
↓
[Per-Person Association via Geometric Utilities]
↓
[Violation Detection per Individual]
↓
[RiskEngine — Timestamped Violation Logging]
↓
[API Response — Number of Persons, Vest violations, Helmet violations]
↓
[Frontend saves first frame of violation and triggers safety alarm on repeated violations]

4.3 Design Decisions

Design Decision	Rationale
YOLOv8 via Ultralytics	Fast inference, easy fine-tuning, built-in NMS
FastAPI backend	Async I/O, lightweight, ideal for frame-by-frame HTTP requests
Confidence threshold = 0.20	Maximizes recall
Geometric association	No secondary ML model needed for person-PPE linking
Modular class design	Detector, RiskEngine, Utils are independently testable
Frontend–Backend Separation	Decouples visualization from processing logic, allowing independent scaling, easier debugging, and flexible deployment environments.

5. DEVELOPMENT OF SYSTEM / PROTOTYPE

5.1 Module 1 — PPEDetector

The PPEDetector class wraps a YOLOv8 model loaded from best.pt. Its detect() method accepts an image array, runs inference at confidence threshold 0.20, and returns a list of detection dictionaries containing class_id, class confidence, coordinates of bounding boxes.

5.2 Module 2 — FastAPI Backend

1. Reads an uploaded image file sent from frontend via device camera
2. Decodes it using OpenCV (cv2.imdecode)
3. Runs detection via PPEDetector
4. Segregates detections by class: persons, helmets, vests, no_helmets
5. Iterates each detected person; associates PPE using utility functions
6. Registers violations in that person's RiskEngine instance
7. Returns violation counts

CLASS_MAP used:
Class ID 0 helmet
Class ID 2 vest
Class ID 6 person
Class ID 7 no_helmet

The backend is packaged as a PyInstaller executable (sys._MEIPASS path resolution), serving a React/Vue frontend from the /dist folder via FastAPI StaticFiles, enabling single-binary deployment on-site without requiring a Python environment.

5.3 Module 3 — RiskEngine

Maintains a deque of violation events per person. Each event stores violation type and Unix timestamp, enabling future time-windowed risk score computation.

5.4 Module 4 — Utility Functions

Handle spatial reasoning to check if helmets lying on the ground or belonging to adjacent workers from being incorrectly assigned.

5.5 Module 5 — Frontend & Integration

The frontend is developed using React, providing a user-friendly interface for real-time PPE monitoring. Its main functionalities include:

- **Camera Input Handling:**

  Supports single or multiple camera inputs.

  Captures video frames at regular intervals (1 frame per second) for processing.

- **Backend Communication:**

  Each captured frame is sent to the FastAPI backend via HTTP POST requests.

  Receives JSON predictions containing PPE detection and violation information.

- **Violation Tracking & Alerts:**

  The first frame of a detected violation is saved for record-keeping.

  Repeats the alert frame until the violation is resolved in subsequent frames.

  Once resolved, a browser-triggered download is initiated, serving as a visual/audio alarm for the user.

- **Single Executable Deployment:**

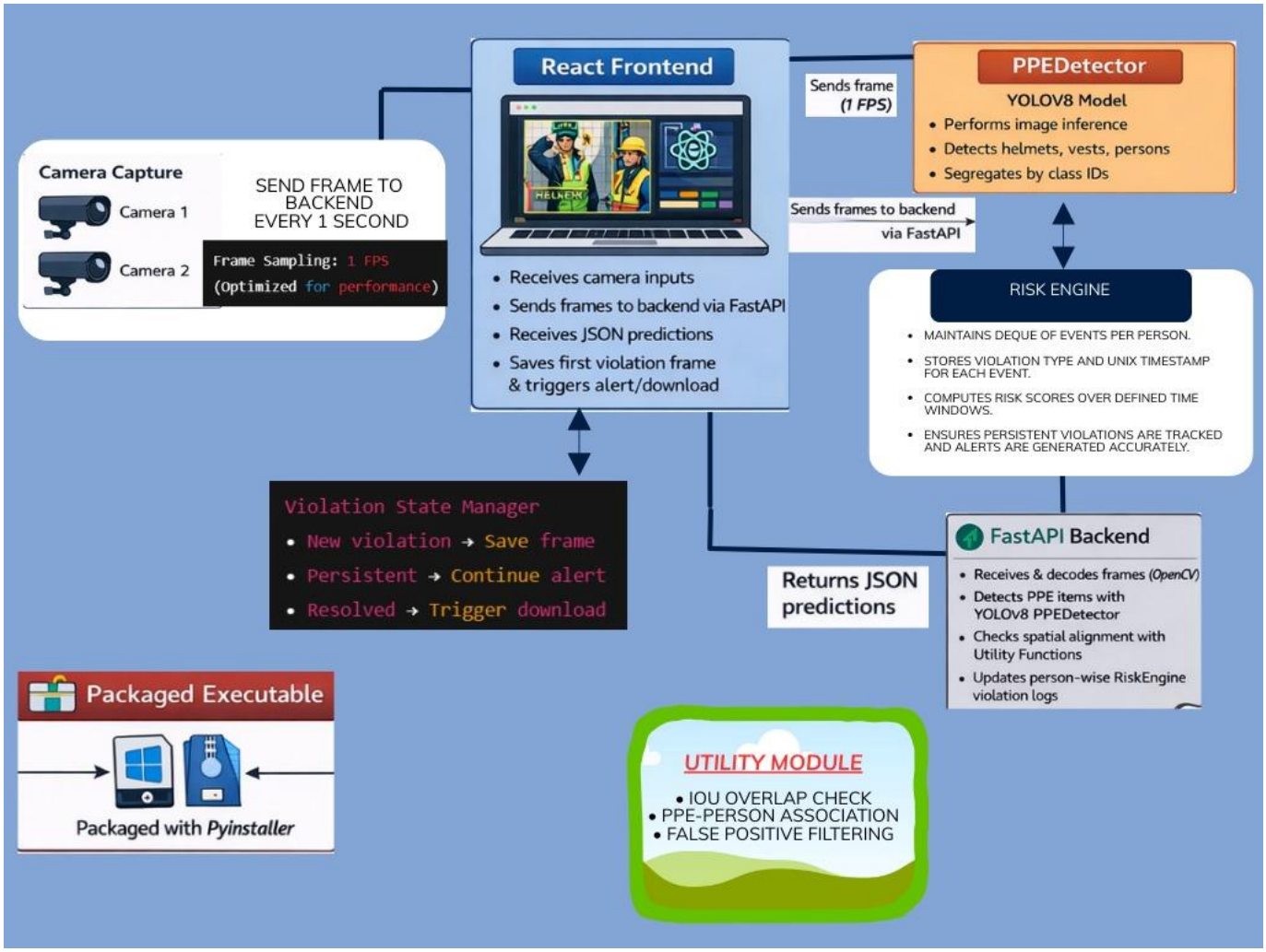
  The React frontend build is served via FastAPI's StaticFiles.

  Using PyInstaller, the backend and frontend are packaged into a single executable, enabling deployment on any Windows machine without requiring Node.js or Python.

  This allows seamless end-to-end operation: from video capture to PPE violation alerts, in a single, portable application.

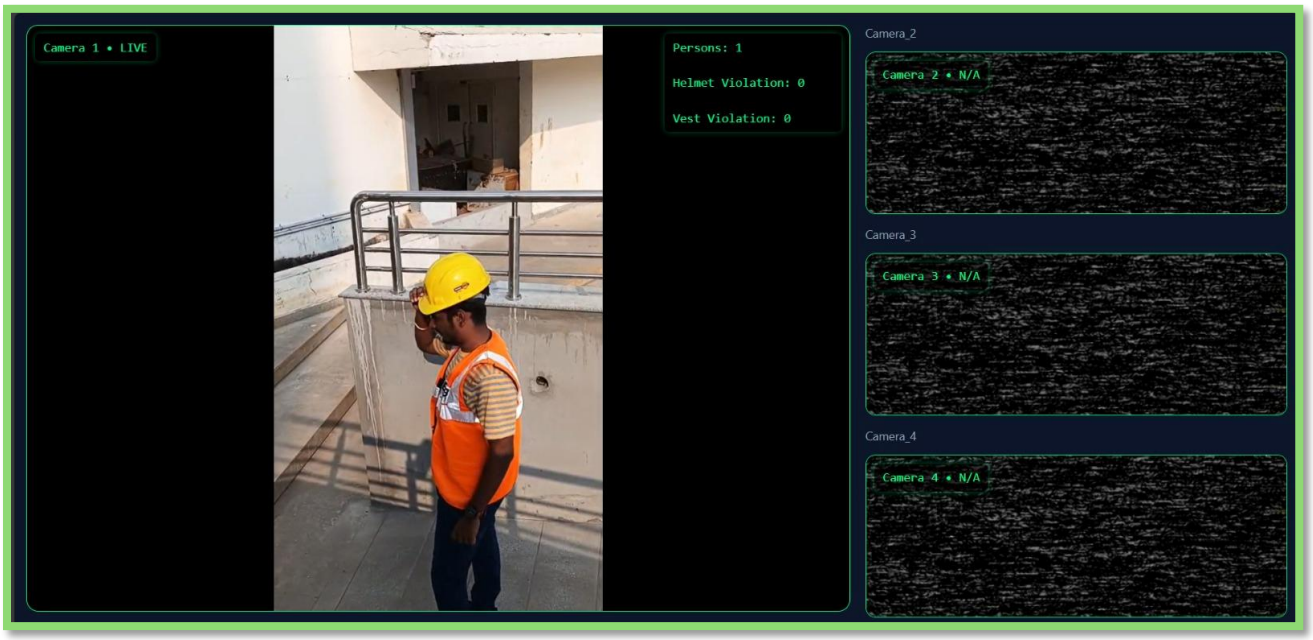
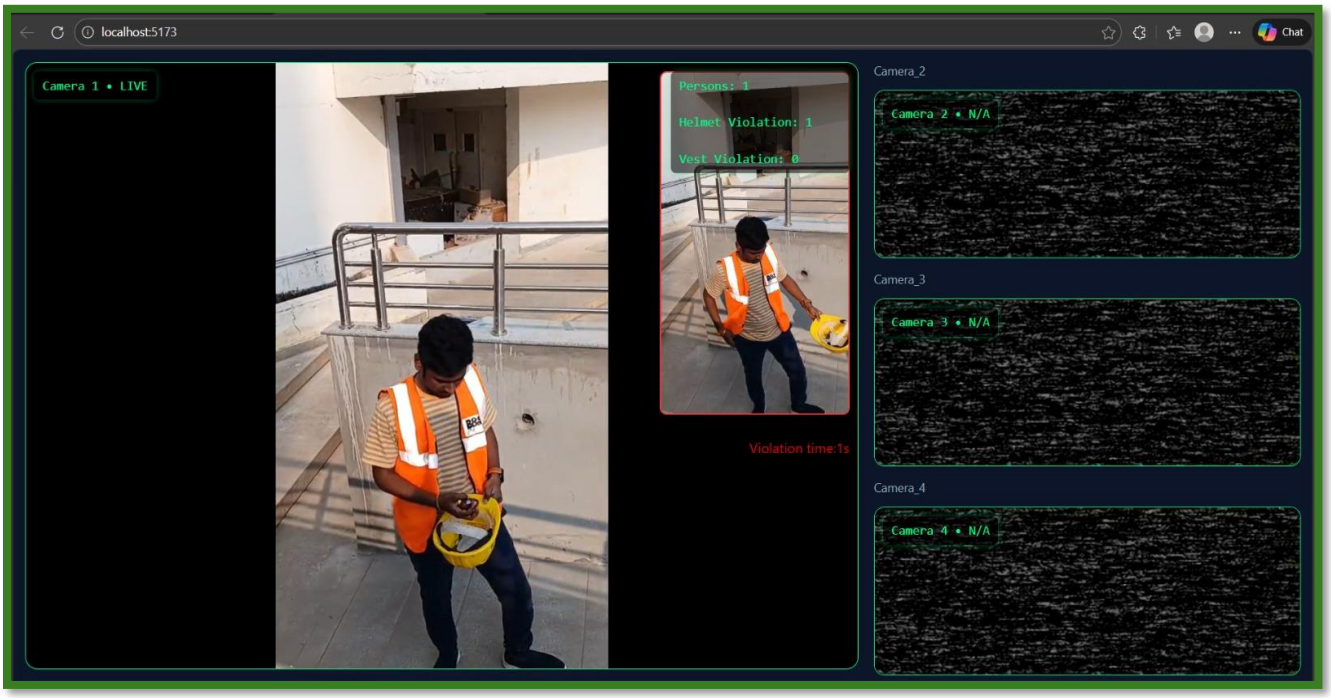
- **User Experience:**
  - Real-time overlay of detected violations ensures instant awareness.
  - Minimal setup required for demonstration or on-site deployment.

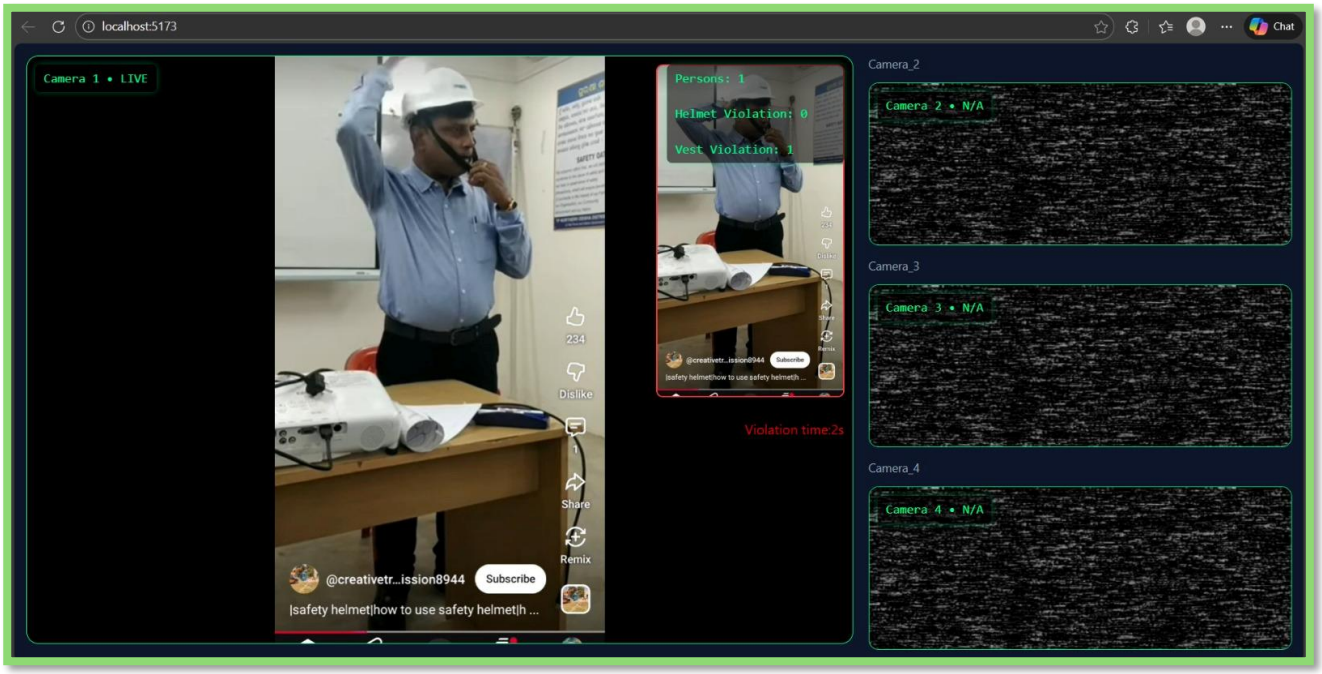
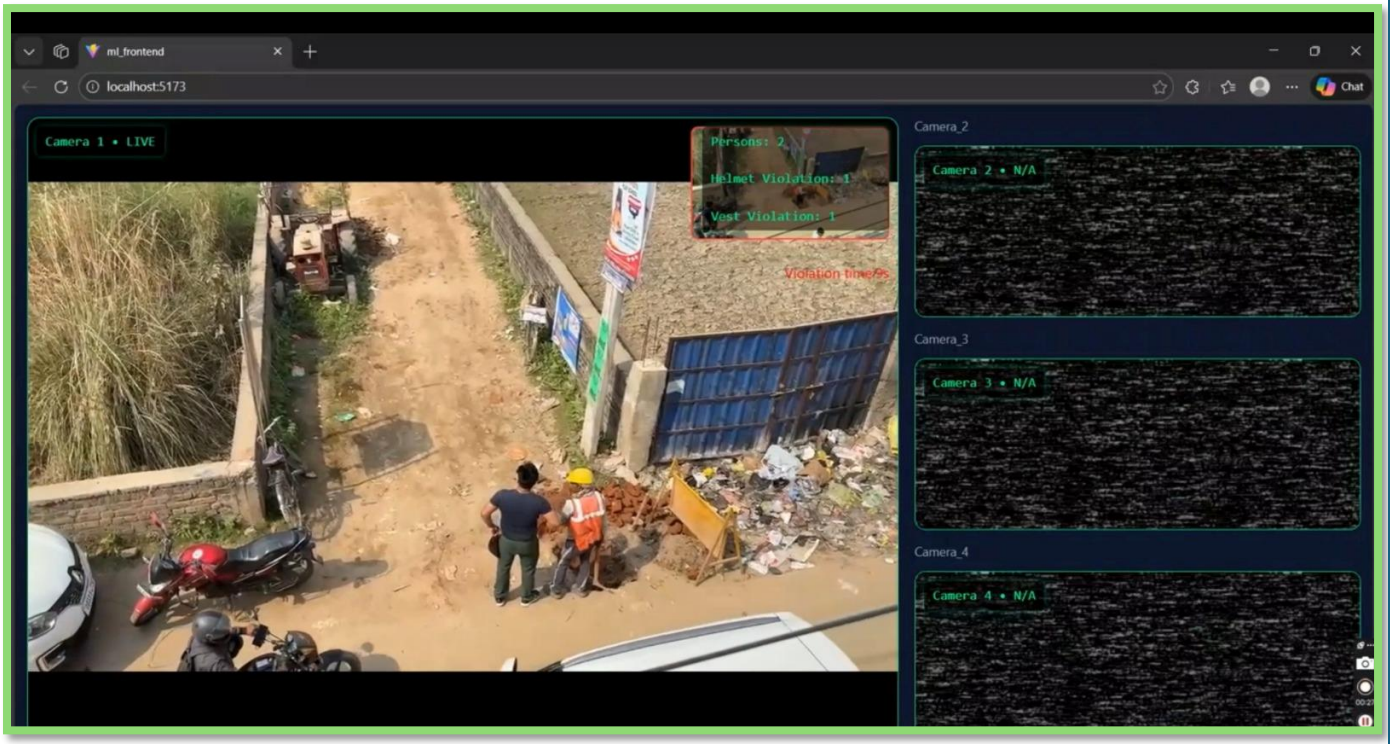
System Architecture — PPE Detection Pipeline



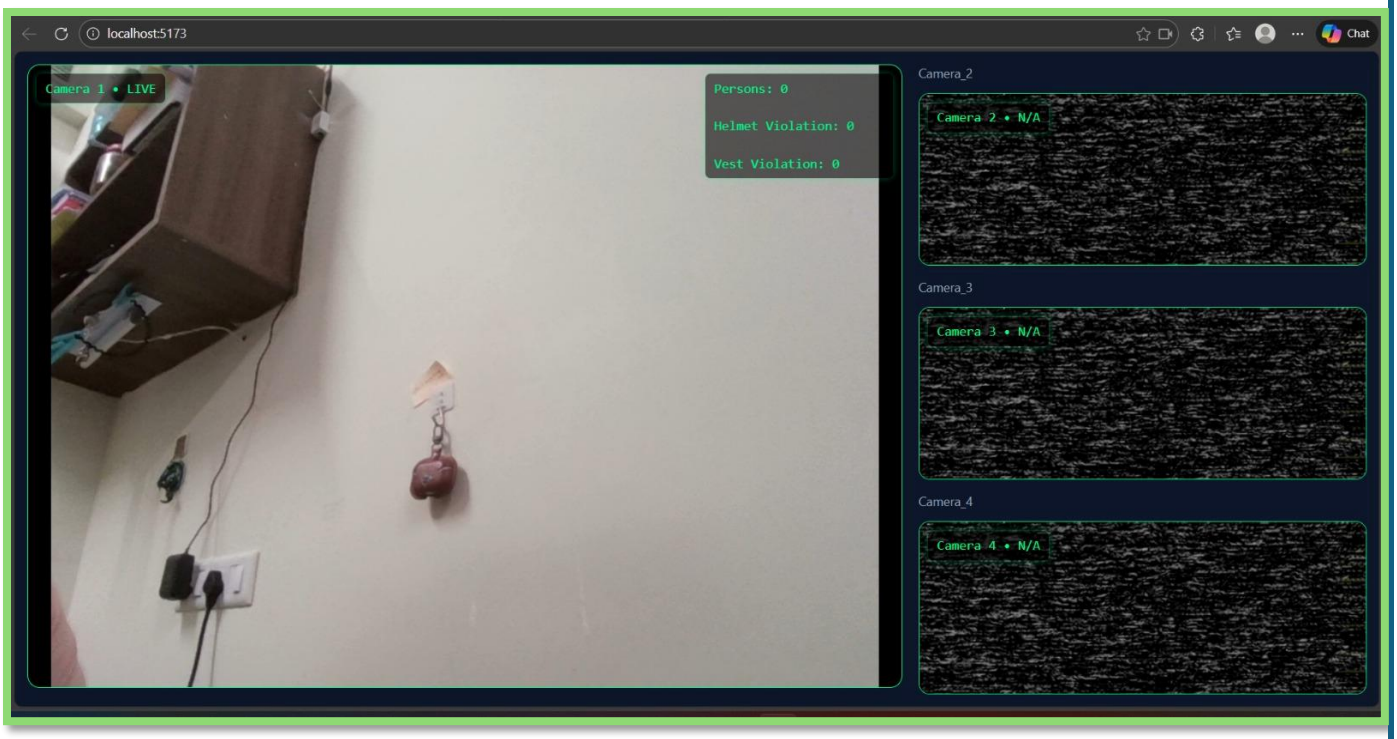
6. DESIGN VALIDATION / PROTOTYPE TESTING

6.1 Test Cases

SR. NO	Test Scenario	Expected Output	Validation Method
1	Single person with helmet and vest	0 violations	static image test
			
2	Single person without helmet	helmet_violation = 1	Static image test
			

3	Single person without vest	vest_violation = 1	Static image test
			
4	Two persons, one compliant, one not compliant	Counts = 1 each violation type	Multi-person static image test
			

5	Empty frame (no persons)	All counts = 0	Edge case test



6.2 Geometric Association Validation

The geometric function was validated against scenarios where two workers stand adjacent. The upper-half centroid constraint successfully rejects helmets whose centers fall in the lower body region of an adjacent person's bounding box in the majority of standard-angle camera positions.

6.3 Known Limitations Identified During Testing

1. Index-based person ID (person_0, person_1) resets each frame — RiskEngine history does not persist to a real individual across frames.
2. Overhead or oblique camera angles reduce accuracy of the upper-half helmet association classification.

6.4 Performance Evaluation

Metric	Value	Notes
Avg Inference Time	**20–60 ms**	**Measured on test device**
Input Frame Rate	**1 FPS**	**Configured frontend capture rate**

Metric	Value	Notes
First Frame Inference	120-150ms(GPU), 360-450ms(CPU-only)	Model warm-up + initialization overhead
Processing Capability	14–16 FPS	Based on steady-state time
Avg API Response Time	105 ms	includes processing
Model Size	5275 kb	(trained YOLOv8 weights file)

7. RESULTS AND DISCUSSIONS

7.1 API Response Structure

For each frame submitted to /analyse, the system returns:

```
{
 "no_of_person": 3,
 "no_of_helmet_violation": 2,
 "no_of_vest_violation": 1
}
```

7.2 Detection Accuracy Observations

YOLOv8, fine-tuned on PPE-domain data with a confidence threshold of 0.20, achieves high recall at the cost of precision. In controlled testing with clear sightlines, helmet detection accuracy was strong. Vest detection showed occasional false negatives when workers wore dark or non-standard colored vests.

7.3 Discussion

The system successfully demonstrates the core pipeline: frame intake → YOLO inference → per-person PPE assessment → violation reporting. The modular design separates concerns cleanly, enabling independent improvement of each component. The primary architectural gap stable cross-frame person tracking is acknowledged and planned for the next development phase. The Risk Engine class is responsible for maintaining a per-person violation history. Each instance is assigned to one detected worker and records every PPE violation that worker commits along with the exact Unix timestamp of the event.

7.4 Frontend Response Behaviour

The frontend interface correctly interprets API responses and visualizes violations in real time. Upon receiving detection results, the interface updates the display with violation indicators and stores the first evidence frame. Repeated violations trigger alert behavior, ensuring that safety supervisors are immediately notified of non-compliance events. The frontend therefore functions not only as a visualization layer but also as an active monitoring interface.

7.6 Deployment Outcome

The packaged executable successfully launched and operated on a separate system without requiring Python or Node.js installations. This validates the portability of the system and confirms its suitability for on-site industrial deployment where technical setup may be limited

8.FUTURE SCOPE

Module: Worker Identification System

The proposed system focuses on real-time detection of safety violations such as missing helmets or harnesses. While identifying the individual responsible for a violation may appear desirable, implementing reliable identity recognition in construction site environments presents significant technical and practical challenges.

Challenges in Real-World Identification:

- Occlusion due to PPE-
  - Helmets, masks, and safety gear obscure facial features, reducing the reliability of face-recognition models.
- Low-Quality CCTV Footage-
  - Construction sites typically use surveillance cameras with: low resolution motion blur variable lighting angled viewpoints These conditions drastically reduce biometric accuracy.
- Uncontrolled Subjects-
  - Sites may contain: visitors, contractors, delivery, personnel and unauthorized individuals making database-based identification unreliable.
- Wearable Tag Limitations-
  - Systems using RFID, QR badges, or body tags depend on workers consistently wearing them. In real violation scenarios, individuals may: forget remove tamper with or intentionally avoid wearing identifiers.

Modular Future Integration

Camera → Detection → Violation Logger → Alert→ Identification Module (Future Integration)

The system architecture is designed to support an optional identification module that can be integrated later if required.

- Possible implementations include:
  - facial recognition API
  - access-control database matching
  - RFID badge integration
  - entry gate attendance logs

This modular approach ensures scalability without affecting current performance.

9. CONCLUSION

This project delivers a functional, modular, AI-powered PPE compliance detection system capable of real-time operation on industrial site camera feeds. Using YOLOv8 for multi-class detection, FastAPI for scalable API exposure, geometric utilities for accurate person-PPE association, and a RiskEngine for violation logging, the system addresses the core challenge of automated safety enforcement at scale.

The system reduces reliance on manual supervision, enables automated violation logging, and provides a foundation for real-time alert generation. Identified limitations — including index-based person tracking and an incomplete risk scoring output — represent clear, planned improvements for the production phase.

10. PROJECT ARTIFACTS & ACCESS LINKS

To ensure reproducibility, demonstration, and evaluation of the proposed system, all project components are provided below.

9.1 Source Code Repositories

- Frontend Repository:
  - https://github.com/TarunSagarSah/tn_impact_frontend.git
- Backend Repository:
  - https://github.com/R-PAl-AIhub/CONSTRUCTION-PPE-DETECTION.git
- Deployed Frontend (Static Preview Only — Backend Not Connected):
  - ml_frontend

9.2 Demonstration Videos

- Demo Video 1:
  - https://youtu.be/gSikuVNupWM
- Demo Video 2:
  - https://youtu.be/zrvYNF5XT34

9.3 Executable Build

A packaged executable version of the system is provided for testing without installing dependencies.

- Download Link (ZIP):
  - https://drive.google.com/drive/folders/1Mh6IURDg4hLgYRlpoS8jp2zb3SqvBxwS?usp=sharing

1) Instructions:
   i) Extract the ZIP file
   ii) **Navigate to:** CONSTRUCTION-PPE-DETECTION-main→dist→run→run.exe
   iii) Run **run.exe** (≈ 48 MB)

9.4 System Requirements

• OS: Windows 10+

• RAM: 8 GB

• CPU: Intel i5 or equivalent

Recommended:

• Dedicated GPU (NVIDIA RTX series recommended for optimal performance and inference speed)